

ME5418: Machine Learning in Robotics

Dynamic Object Manipulation with a 7-DoF Arm (Franka Emika Panda)

Kho Kah Chun



Department of Mechanical Engineering
National University of Singapore

AI Declaration

I used GPT-5 to generate an empty structure of the report and check the grammar.
I am responsible for the content and quality of the submitted work.

Abstract

This project studies dynamic object manipulation using a 7-DoF Franka Emika Panda arm equipped with a rigid tray attached to the wrist. The task requires transporting a cylinder across a workspace while preventing toppling or dropping due to inertial and contact dynamics. We formulate the problem as a continuous-control reinforcement learning (RL) task and implement a MuJoCo-based environment with configurable dynamics (friction, mass, restitution) and curriculum phases. Our primary agent is a PPO-based actor-critic with a multi-layer MLP backbone. Training uses VecNormalize and curriculum scheduling. Results show the learned policies reliably move the tray to target poses while reducing toppling/drop events compared to naive trajectory-following baselines; we report learning curves, success rates, and ablations on observation modalities and curriculum. We conclude by discussing limitations (sample efficiency and sim-to-real gap) and propose future directions including sensor-based cylinder observation and transfer learning for real-robot deployment.

Contents

1	Introduction	5
2	Related Work	5
3	Methodology	6
3.1	OpenAI Gym Environment	6
3.2	Observation and Reward Structure	6
3.3	Learning agent and algorithm	7
3.4	Evaluation callback, logging and checkpointing	7
3.5	Training procedure and reproducibility notes	7
4	Results	8
4.1	Training Scenarios	8
4.1.1	Quantitative results	8
4.2	Test scenarios (what each phase represents)	9
4.2.1	Qualitative results	9
4.3	Interpretation	9
5	State-of-the-Art Conventional Alternative — STOMP	9
5.1	Short comparison between RL and STOMP	10
6	Discussion	10
7	Conclusion	11
A	TensorBoard visualizations and training diagnostics	12
B	STOMP Methodology	15
B.1	Brief pipeline (implementation highlights)	15
B.2	Constraints (implementation)	16

1 Introduction

Robotic manipulation frequently involves carrying or manipulating objects that have internal degrees of freedom or dynamic responses to the manipulator’s motion (e.g., liquids, rolling parts or loosely constrained payloads). Traditional pose-based planning and inverse-kinematics (IK) controllers plan kinematic trajectories but typically ignore the payload’s internal dynamics, which can lead to undesirable events such as sloshing, shifting or toppling. These failure modes are important in real-world applications — for example, mobile service robots carrying liquids or delicate components must avoid spills or misalignment during motion.

This project focuses on a representative dynamic manipulation problem: transporting a cylindrical object on a tray attached to a Franka Emika Panda arm without toppling or dropping the cylinder. We cast the task into the reinforcement learning (RL) framework where the agent controls joint increments in continuous action space and receives rewards that encourage timely and stable transport of the cylinder to a goal pose. The primary objectives are:

- Design a physically realistic MuJoCo environment with configurable object dynamics, observation noise and curriculum phases.
- Train RL agents (PPO and variant actor-critic networks) to achieve robust tray-pose control under disturbances and model variations.
- Evaluate agent performance and study the impact of observation choices, network architecture and curriculum.
- Compare with another state-of-the-art approach in dynamics transporting.

2 Related Work

Dynamic payloads and manipulation of non-rigid or internally mobile objects are active research areas. Classical model-based methods (e.g., inverse kinematics with time-scaling, LQR, Model Predictive Control) focus on planning and tracking geometric trajectories while often assuming a rigid payload. Such methods can fail when payload dynamics (sloshing, rolling, frictional sliding) significantly affect the object state.

Recent work has applied learning-based methods to these tasks. Reinforcement learning (RL) has shown success in continuous control tasks and can implicitly learn feedback strategies that account for unmodelled dynamics. Actor-critic methods (PPO, SAC, A2C variants) are commonly used for continuous robotic control. Curriculum learning and observation normalization are effective aids for stabilizing training and improving generalization. There is also growing interest in combining model-based controllers (e.g., PD or MPC) with learned policies for improved safety and sample efficiency. Our work builds on these ideas, as we keep a low-level PD actuator model for stability and seek a learned policy that reasons over both arm and payload dynamics.

3 Methodology

3.1 OpenAI Gym Environment

We implemented a MuJoCo-based Gym environment (`TrayPoseEnv`) that models a Franka Emika Panda with a wrist-mounted tray and a free cylinder. The environment and training pipeline are driven by `envs/traypose/config.yaml` and `train_traypose_tensorboard.py`. Key implementation and default parameters are:

- **Action space:** 7-D normalized joint angle increment commands in $[-1, 1]^7$, de-normalized by `max_speed` (default 0.3) and integrated to a desired joint target `q_des`. Commands are rate/torque limited by `delta_tau_max` and enforced via a PD position controller with gravity compensation.
- **Low-level control:** PD gains and limits are specified in `config.yaml`. Solver control uses `substeps=10`.
- **Stabilization / resets:** each reset runs a stabilization loop of `stabilization_steps` (default 100) to let the PD controller and any floating objects settle before agent control begins.
- **Curriculum:** a configurable multi-phase curriculum (default `num_phases=5`) uses a linear progression scheme. Per-phase parameters (goal pose, yaw tolerances, per-phase rewards/penalties and `max_success_threshold`) are read from `config.yaml` and synchronized between training and evaluation via the custom evaluation call-back.
- **Config-driven:** environment physics and penalties (friction/mass ranges, drop/-topple thresholds, sliding/slant zones, per-step penalties and success rewards) are externalized in `config.yaml` for reproducibility and ablation studies.

3.2 Observation and Reward Structure

Observations and rewards are implemented:

- **Observation vector (default 36-D):** joint angles q (7) and velocities $\dot{q}$ (7); tray pose (pos 3 + RPY 3) and tray linear/angular velocities (3+3); goal pose (x, y, z, yaw) (4); cylinder-relative horizontal position (x, y) and velocities ($\dot{x}, \dot{y}$) (2+2), plus cylinder tilt and tilt rate (2). By default observation noise is zero.
- **Reward composition:** per-step base penalty plus progress reward, sliding and slant (tilt) penalties (soft hinge-style between configured min/max magnitudes), large hard penalties for drop and topple, and a per-phase hold/success bonus. Drop, topple, hold and success reward are increasing along the phases. Episodes terminate on drop, topple, success or timeout (truncation).

3.3 Learning agent and algorithm

Training uses Stable-Baselines3. The training script supports PPO and SAC; our standard experiments use PPO with the following explicit defaults (also configurable via CLI):

- **Policy & architecture:** PPO with `MlpPolicy`, policy network `net_arch = [256,256]` (specified via `policy_kwargs`).
- **PPO defaults:** `learning_rate = 3e-4`, `n_steps=2048`, `batch_size=256`, `n_epochs=10`, `gae_lambda=0.95`, `clip_range=0.2`, `ent_coef=0.0`, `vf_coef=0.5`, `max_grad_norm=0.5`, discount γ default = 0.9995.
- **Normalization:** optional `VecNormalize` (CLI flag `-normalize`). When enabled training uses `norm_obs=True`, `norm_reward=True` and clipping (`clip_obs=10`). The eval env uses the same normalization wrapper (with reward normalization disabled for eval env).
- **Environment and seeds:** training uses `DummyVecEnv` (single process by default). Each env is created via `make_env(...)` which calls `env.reset(seed=seed)` for reproducibility. Eval env seeds are offset by +1000.

3.4 Evaluation callback, logging and checkpointing

A custom `CustomEvalCallback` in `train_traypose_tensorboard.py` performs deterministic evaluation and curriculum synchronization:

- **Deterministic eval:** default `eval_freq=5000` timesteps, `n_eval_episodes=10`; actions during eval are chosen with `deterministic=True`.
- **Metrics logged to TensorBoard:** eval mean reward, mean episode length, success/drop/topple/truncated rates, mean cylinder angle/offset, curriculum mean phase, consecutive successes and success threshold are saved under the training run log directory.
- **Phase synchronization and checkpointing:** the callback synchronizes the eval env's `current_phase` with the training environment and saves a best model per curriculum phase.
- **Diagnostics:** the callback records per-eval-episode info fields (`is_success`, `terminated_due_to_drop/topple`, cylinder metrics) facilitating the TensorBoard plots included in the Appendix A

3.5 Training procedure and reproducibility notes

The training entry point (`train_traypose`) creates logs and saves CLI args. Relevant default CLI arguments and notes:

- **Default CLI values:** `algorithm=PPO`, `total_timesteps=20000` (CLI-configurable), `learning_rate=3e-4`, `eval_freq=5000`, `n_eval_episodes=10`, `seed=0`.
- **Reproducibility:** set `-seed` and optionally `-normalize` to reproduce runs; Vec-Normalize statistics and final model artifacts are saved to the run log directory.
- **Recommended experiment metadata to report:** total timesteps per run, number of independent seeds, deterministic eval episodes per checkpoint, whether normalization was used and the exact `config.yaml` values (PD gains, `stabilization_steps`, per-phase success thresholds and penalty magnitudes). These values are already captured automatically in the run directory via saved `args.json` and checkpoint files.

4 Results

We evaluate agents with the `CustomEvalCallback` and report learning curves and deterministic evaluation metrics logged to TensorBoard. Below we summarize the quantitative findings, point the reader to the plots in the Appendix, and give brief interpretation.

4.1 Training Scenarios

There are 2 sets of trainings:

- one which starts from phase 1 and keeps advance to the next phase, but ends at phase 4 due to the total timesteps.
- one which starts directly from phase 5 but trained for lesser total timesteps.

4.1.1 Quantitative results

- **Curriculum progression:** See Figure 2 (phase1–4) and Figure 3 (phase5). The TensorBoard `curriculum/mean_phase` and `curriculum/consecutive_successes` plots indicate when the curriculum advanced and where success counters collapsed.
- **Training diagnostics:** See Figure 4 and Figure 5. Monitor explained variance, value loss and approx-KL for spikes or sustained instability as indicators of unstable updates or poor normalization/hyperparameter choices.
- **Evaluation metrics:** See Figure 6 and Figure 7. We report success rate, drop rate, topple rate, mean final cylinder offset and mean episode length. Each metric in the table is averaged over `n_eval_episodes` deterministic eval episodes per checkpoint.
- **Rollouts (generalization):** Compare side-by-side rollout traces in Figures 8 and 9: the phase1–4 run advances through the curriculum but exhibits a brittle region around phase 3; the phase5 run attains high stochastic training success but inconsistent deterministic rollouts.

4.2 Test scenarios (what each phase represents)

- Phase 1 to 4 — the displacement increases along the phase.
- Phase 5 — furthest goal position with a yaw constraint.

The task is separated into multiple phases in order to guide the agent slowly approaching the goal pose.

4.2.1 Qualitative results

- By testing the performance of best model (`PPO_best_phase_4.zip` for each phase, we see that the model able to solve each phase. However, the model seems to be not learnt to stop around the goal position (please refer to `RL_Phase3.webm`).
- Evaluation reveals consistent, interpretable behaviours: policies learn anticipatory wrist tilting to recenter the cylinder and apply cautious slowdowns before tight motions (see Figure 6 and Figure 7). Common failure modes are likewise clear as rapid accelerations that induce sliding or cause the cylinder to topple.
- Performance is phase-dependent: policies trained for phases 1, 2 and 4 produce sensible corrective actions and succeed in their nominal conditions, whereas the phase-3 policy repeatedly fails; this brittleness is visible in the curriculum progression (Figure 2) and the low phase-3 evaluation success (Figure 6).

4.3 Interpretation

- Phase3 is a brittle region: this may because of overfitting due to the higher number of consecutive success needed to advance to next phase.
- Phase5 shows probable overfitting / reliance on stochastic training signals (Figure 5).

5 State-of-the-Art Conventional Alternative — STOMP

We implement STOMP as a practical, sampling-based trajectory optimizer using the provided MATLAB utilities. The top-level driver is `panda_tray_STOMP_Path_Planning.mlx`, which sets up parameters and launches `helperSTOMP.m` to orchestrate sample generation, cost evaluation and trajectory updates. Per-trajectory costs and safety/physics constraints are implemented in `stompTrajCost.m` (task-specific penalties).

For more information about the implementation of STOMP, please refer to Appendix B.

With the methodology mentioned, STOMP is able to find a smooth trajectory that keep the cylinder upward and has minimum linear/angular acceleration (Figure 1).

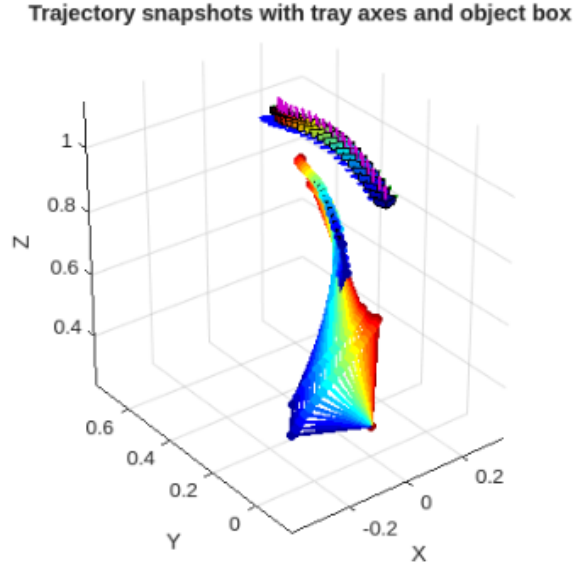


Figure 1: Trajectory Plot of STOMP

5.1 Short comparison between RL and STOMP

- STOMP (sampling-based trajectory optimization) produces deterministic, safety-aware reference trajectories with explicit penalties, but lacks reactive feedback and requires careful cost tuning and online optimization.
- The learned RL policy provides reactive closed-loop corrections that compensate for unmodelled disturbances, but at the cost of high sample complexity, some training instability, and reliance on reward structure.

6 Discussion

Our experiments show that curriculum learning and a twin-critic actor-critic architecture can produce robust, phase-specific behaviours in simulation, but also reveal brittle regions and training instabilities.

Key observations

- The phase1–4 run advances through the curriculum and yields reliable policies for phases 1, 2 and 4; phase 3 is a consistent failure region (see Figures 2, 6).
- The phase5 run exhibits high stochastic training success but poor deterministic roll-out generalization, indicating overfitting or reliance on stochastic/privileged signals (Figures 5, 9).
- Training diagnostics (explained variance, value loss, approx-KL) show spikes and high variance in both runs, pointing to unstable updates or sensitivity to normalization/hyperparameters (Figures 4, 5).

Practical lessons

- Always validate with deterministic rollouts and multiple seeds: high stochastic eval scores can mask brittle deterministic behaviour.
- Curriculum progression exposes narrow failure regions; per-phase evaluation and observation-distribution logging are critical to diagnose mismatches.

Future works

- Increase the control timestep of the agent from 0.001 s to 0.05 s which is more realistic to the actual robotics actuation and also less sensitive to noise.
- Replace direct cylinder observations with realistic sensors: camera-based pose estimators or torque/force sensors. Evaluate how partial/noisy observations affect policy performance.

7 Conclusion

We presented an RL pipeline for dynamic tray transport using a 7-DoF Franka Emika Panda and showed that curriculum learning and a twin-critic actor-critic policy can learn reactive behaviours that reduce sliding and toppling in simulation. Policies for phases 4 perform well in all nominal regimes.

Immediate next steps are to (1) diagnose and repair the phase-3 failure region, (2) reduce overfitting in the phase-5 run via stronger deterministic evaluation and observation regularization, and (3) replace privileged inputs with realistic sensing and adopt hybrid model-learned control before attempting hardware deployment. Longer term, we will pursue transfer learning and domain randomization.

A TensorBoard visualizations and training diagnostics

The figures below show TensorBoard logs from the two training runs referenced in the main text.



Figure 2: Phase1–4: curriculum progression, consecutive success counters and thresholds.

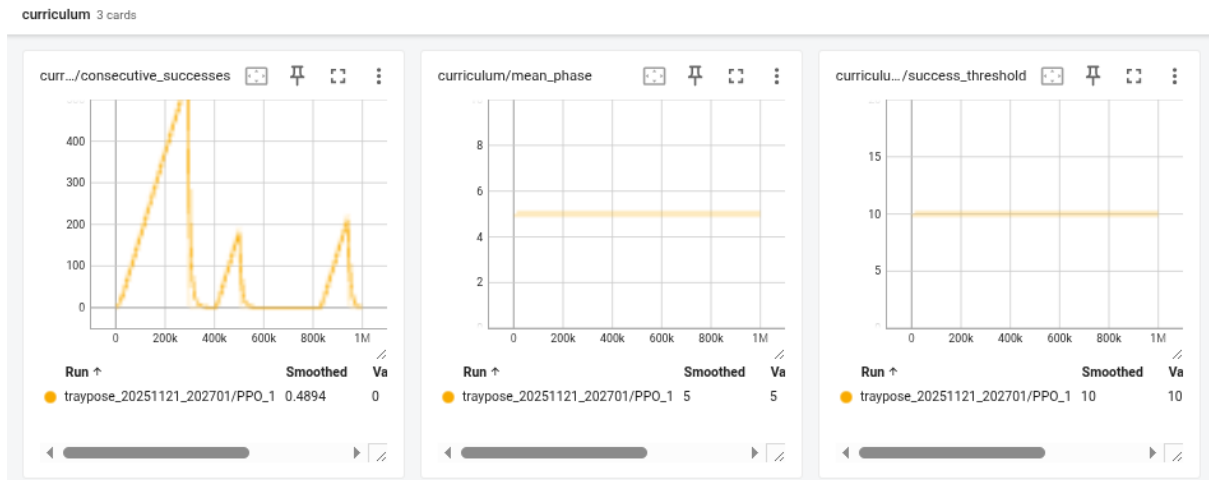


Figure 3: Phase5: curriculum progression, consecutive success counters and thresholds.

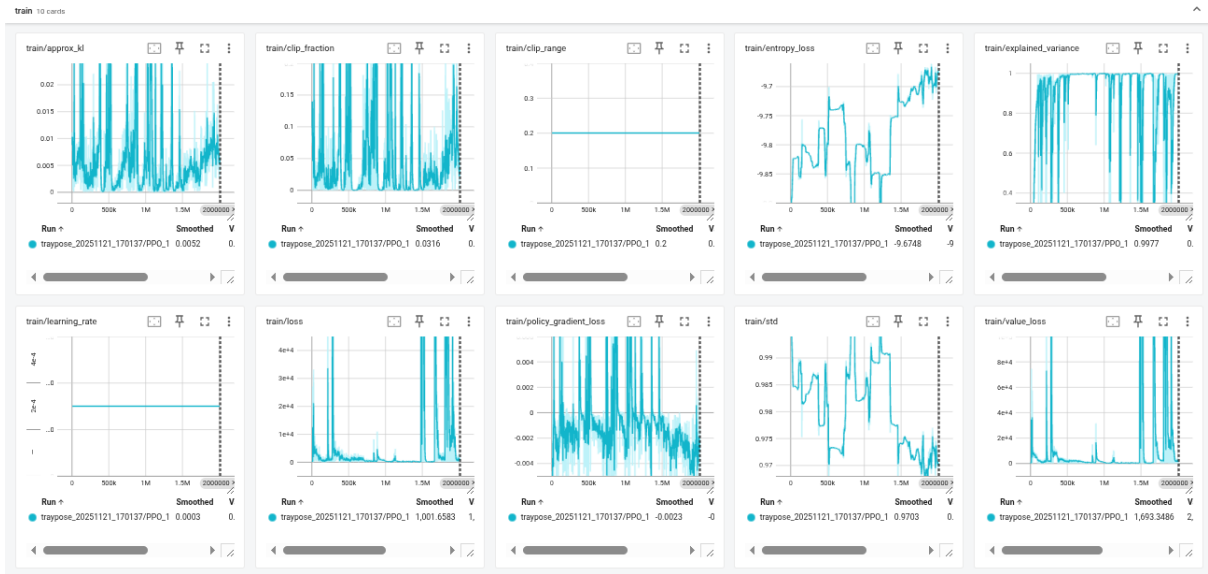


Figure 4: Phase1-4: training diagnostics (losses, KL, value-loss, explained variance).



Figure 5: Phase5: training diagnostics (losses, KL, value-loss, explained variance).



Figure 6: Phase1-4: deterministic evaluation metrics (success/drop/topple rates, cylinder stats).

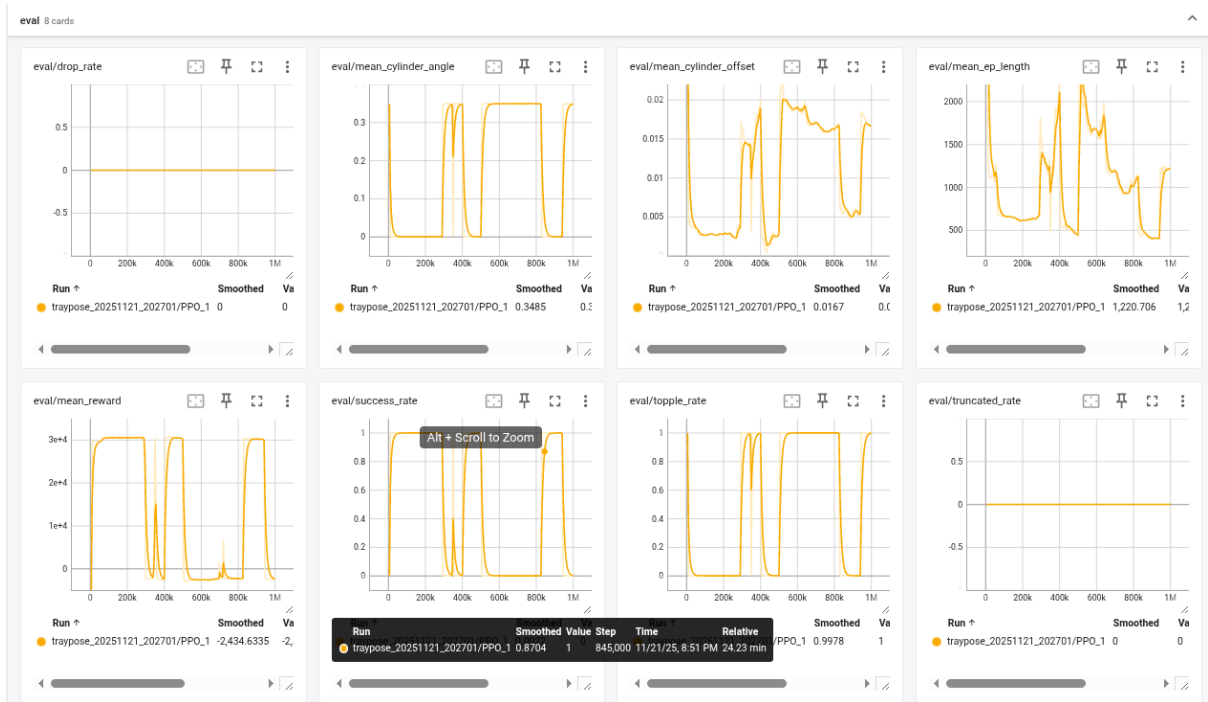


Figure 7: Phase5: deterministic evaluation metrics (success/drop/topple rates, cylinder stats).

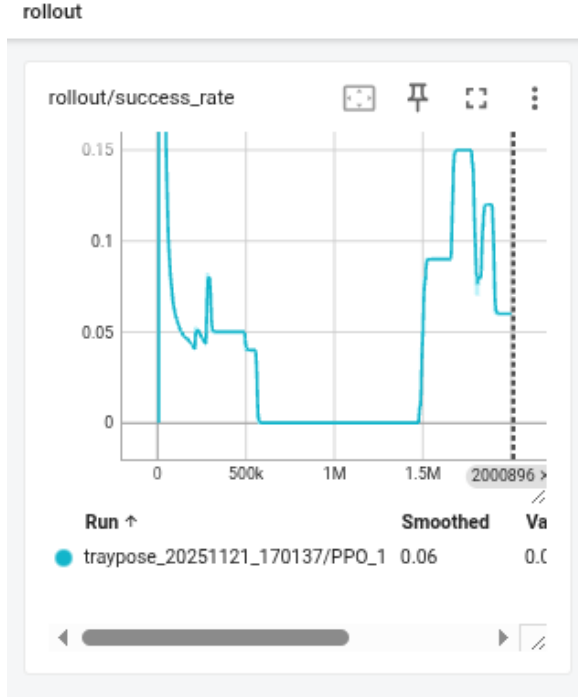


Figure 8: Phase1–4: rollout success rate across checkpoints.

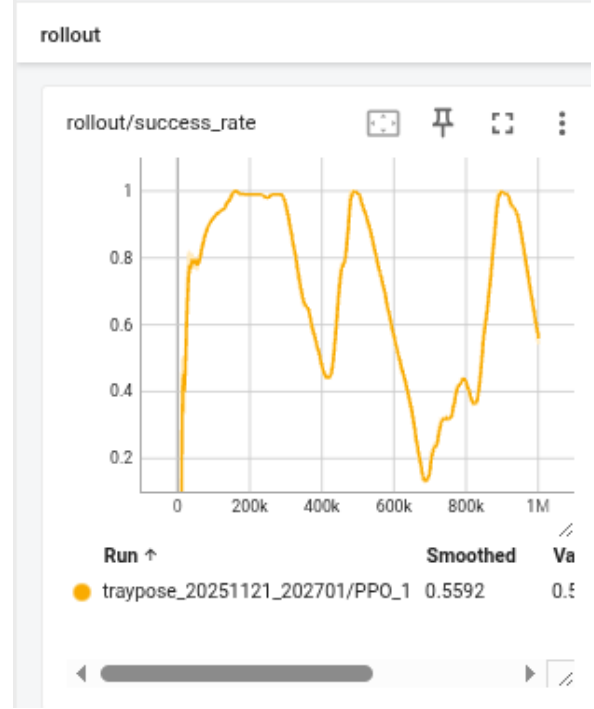


Figure 9: Phase5: rollout success-rate across checkpoints.

B STOMP Methodology

B.1 Brief pipeline (implementation highlights)

- **Trajectory parametrization:** joint-space waypoints $\mathbf{q}_{0:N}$ are updated by `stompUpdateTheta.m`. Per-step increments and smoothness derivatives are computed with `stompDTheta.m`.
- **Sampling and update:** `stompSamples.m` draws M Gaussian perturbations per iteration; the probabilistic update uses `stompUpdateProb.m` to compute weights $w^{(m)} \propto \exp(-C^{(m)}/\lambda)$ and apply the cost-weighted correction.
- **Kinematics & approximate dynamics:** forward-kinematics and waypoint-to-end-effector transforms are computed with `updateJointsWorldPosition.m`; finite-difference estimates of end-effector/cylinder linear and angular accelerations (used in sliding/tipping penalties) are derived from these kinematic traces.
- **Obstacle / geometry checks:** `sEDT_3d.m` provides signed-distance transforms and `stompRobotSphere.m` performs simple robot-sphere collision probes; their outputs feed `stompObstacleCost.m`.
- **Cost structure:** `stompTrajCost.m` composes the scalar cost

$$C = C_{\text{smooth}} + C_{\text{goal}} + C_{\text{orient}} + C_{\text{slide}} + C_{\text{tip}} + C_{\text{control}} + C_{\text{coll}},$$

where the stability-related terms ($C_{\text{orient}}, C_{\text{slide}}, C_{\text{tip}}$) are implemented as soft penalties (smooth hinges) inside `stompTrajCost.m`.

B.2 Constraints (implementation)

To achieve the stable transportation, the movement of the cylinder is constrained in 3 ways - Orientation, sliding and tipping.

Orientation: the cylinder rotation R_{cylinder} is penalised by

$$\text{qc_orientation}(\mathbf{i}) = \|R_{\text{cylinder}}(:, 3) - [0, 0, 1]^\top\|^2,$$

to avoid deviation of the cylinder z-axis from world z.

Sliding: linear acceleration is estimated by central differences

$$a_{\text{world}} = \frac{p_{t+1} - 2p_t + p_{t-1}}{\text{dt}^2}, \quad a_{\text{cylinder}} = R_{\text{cylinder}}^\top a_{\text{world}}.$$

Let $a_{\text{tan}} = \|a_{\text{cylinder}}(1 : 2)\|$ and $a_n = a_{\text{cylinder}}(3)$. Using friction μ we form the soft violation

$$v_{\text{slide}} = \max(0, a_{\text{tan}} - \mu(g - a_n)_+),$$

and store $\text{qc_sliding}(\mathbf{i}) = v_{\text{slide}}^2$.

Tipping: angular acceleration is approximated from Euler angles by a central second difference

$$\alpha \approx \frac{e_{t+1} - 2e_t + e_{t-1}}{\text{dt}^2},$$

with wrapping via `wrapToPi`. Roll/pitch components exceeding `angAccLimit` produce a hinge penalty stored in `qc\_tipping(i)`.